

SD Basic - Sequential Files

Reading and writing ordinary operating-system files a line or a block at a time.

A sequential file is an ordinary file on disk, read and written a line at a time rather than a record at a time. It is how SD exchanges data with anything that is not SD — a log, an export, a file dropped in a folder by another system.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Opening

```

openseq file.name, record.id {append | overwrite | readonly} to file.variable
  then statements
else statements

openseq pathname {append | overwrite | readonly} to file.variable
  then ... else ...

```

The two-name form addresses a record inside a **directory** file — an SD file whose records are ordinary files in a folder. The one-name form takes an operating system path and reaches anything on the machine the account is allowed to read.

The `then` and `else` branches do not mean success and failure.

`then` the file **already existed**

`else` it did **not** exist, and has been created — **or** the open genuinely failed

`openseq 'ZZDIR', 'LINES'` on a name that does not exist takes the `else` branch and gives a usable file variable.

So `else` is not an error branch, and a program that writes `stop 'cannot open'` there will refuse to create a file it was supposed to create. To tell "created" from "failed", test the variable — a failed open leaves it unusable and the next `writeseq` takes *its* `else` branch.

KEYWORD

append position at the end, so writes add to what is there

KEYWORD

overwrite	truncate on open
readonly	open without taking a write lock

Reading

```
readseq variable from file.variable {on error statements} then ... else ...
readblk variable from file.variable, bytes then ... else ...
```

<code>readseq</code>	one line, without its terminator
----------------------	---

<code>readblk</code>	a fixed number of bytes, terminators included
----------------------	---

`readseq`'s `else` branch is end of file. Three lines written and read back give `3` and the first as `line one`, and `status()` at end of file is **3006** — the same code a missing record gives in SD Basic - File Handling.

```
openseq 'EXPORT', 'DATA' to f else stop 'cannot open'
loop
  readseq line from f else exit
  ...
repeat
closeseq f
```

Writing

```
writeseq variable to file.variable then ... else ...
writeseqf variable to file.variable then ... else ...
writeblk variable to file.variable
weofseq file.variable
```

<code>writeseq</code>	write a line and a terminator
-----------------------	-------------------------------

<code>writeseqf</code>	the same, then flush to disk
<code>writeblk</code>	write bytes with no terminator added
<code>weofseq</code>	truncate the file at the current position

`writeseq` **does not flush**. A program that writes a log and then aborts can lose the tail. `writeseqf` costs a disk write per line, which is why it is a separate statement — use it for anything that has to survive a crash, and `writeseq` plus a `flush` at the end for bulk output.

`weofseq` at the start of a file is how you empty it: `openseq` positions at the beginning, and truncating there discards everything.

THE LINE TERMINATOR IS CRLF

Two lines written with `writeseq` and read back with `readblk`, byte by byte:

bytes	65 66 13 10 67 68 13 10
meaning	A B CR LF C D CR LF

This port writes CRLF, not LF. That is deliberate — a file SD produces opens correctly in Notepad and every other Windows tool, which was not true of the Linux original. `readseq` accepts either, and strips whichever it finds, so a file written on Linux still reads correctly here.

It matters when you count bytes. A file of n lines is n bytes longer than the same file with LF endings, so an offset computed by adding up line lengths must allow two bytes per line, not one.

Position

```
seek file.variable {, offset {, relative.to}}
```

RELATIVE.TO

0	from the start of the file
1	from the current position
2	from the end

`seek sq, 0, 0` followed by `readseq` returns the first line; `seek sq, 5, 0` followed by `readblk sq, 4` returns the four bytes from offset 5.

`seek` with no offset returns to the start. Mixing `seek` with `readseq` is safe, but an offset that lands in the middle of a CRLF gives a `readseq` that starts with a stray terminator.

Buffering and timeouts

```
nobuf file.variable
flush file.variable
timeout file.variable, seconds
```

<code>nobuf</code>	turn buffering off for this file — every write goes straight out
--------------------	--

<code>flush</code>	push what is buffered now
--------------------	---------------------------

<code>timeout</code>	how long a read waits before giving up
----------------------	--

`nobuf` is for a file another process is watching. `timeout` matters on a named pipe or a device, where a read can otherwise block for ever; on an ordinary file it does nothing useful.

Asking about the file

```
status variable from file.variable then ... else ...
```

Fills *variable* with the operating system's view of the file: **21 fields** — size, timestamps, permissions and the rest, in the order the C library reports them.

`fileinfo(file.variable, 5)` reports a sequential file's type as **5**, and key `1006` gives the current position.

Deleting

```
deleteseq file.name, record.id then ... else ...
deleteseq pathname then ... else ...
```

Removes the file. `closeseq` releases the variable without deleting anything; a sequential file left open is closed when the program ends, but the buffered tail is only guaranteed once `closeseq` has run.

Directory files, and where the marks go

A **directory** file is an SD file whose records are ordinary files. That is what makes sequential access to them possible, and it is the usual target for `openseq`.

Because the records are ordinary files, SD has to decide what to do with mark characters on the way in and out. `mark.mapping` controls it — see SD Basic - Dynamic Arrays. With mapping on, field marks become newlines and back again; with it off the bytes pass through untouched, which is what you want for a file another system produced.

What is not here

Nothing in the sequential-file group has been removed from this port. The **VFS** layer, which used to sit between these statements and the operating system, has been removed from the C entirely.

See also

SD Basic - File Handling · SD Basic - CSV Files · SD Basic - Data Conversion.
